Capstone Design [202601-ICE/CSE4020]

Project #1: Multi-Point Maze Navigation

Team: Donatello
Team Members:
U2210088 Sarvar Islamov (Team Leader)
U2210013 Fuzaylkhon Abdurakhimov
U2210077 Gayday Aleksey
U2210086 Inomjonov Javohirbek
U2210099 Kalimullin Rinat
U2210122 Shovkatjon Komilov
U2210251 Javohirbek Xatamov

April 23, 2026

1 Approach and Architecture

1.1 Navigation Framework

Our system is built upon the ROS Navigation Stack, which provides a robust framework for autonomous mobile robots. The architecture integrates environmental sensing, localization, and dual-layer path planning. We utilized `move_base` as the primary coordinator, interfacing with our custom `flag_manager` node to handle high-level mission logic. To comply with the project constraints, we do **not** hardcode any maze-specific waypoints or paths; all target flags are provided at runtime (in our physical run, via RViz point publishing).

1.2 Localization and Perception

To ensure the robot can navigate accurately within the campus maze, we implemented Adaptive Monte Carlo Localization (AMCL). This probabilistic system uses LiDAR scans (`/scan`) and wheel odometry (`/odom`) to estimate the robot's position relative to a static map. Perception is handled by an LDS-02 LiDAR sensor, providing a full 360° field of view for real-time obstacle detection.

1.3 Path Planning and Parameter Tuning

The navigation logic is split into two planners:

- **Global Planner (Navfn):** Generates a collision-free path from the current pose to the flag using the A* algorithm.

- **Local Planner (DWA):** Computes velocity commands (`cmd_vel`) by simulating trajectories in a short time horizon.

To prevent "wall-hugging" and freezing, we performed extensive parameter tuning:

- `inflation_radius`: Increased to 0.55m to force the robot to stay centered in corridors.
- `cost_scaling_factor`: Adjusted to 10.0 to create a steeper cost gradient near walls.
- `occdist_scale`: Increased to 0.1 to penalize trajectories that pass too close to obstacles.

1.4 Flag Selection and Scoring Algorithm

The `flag_manager` node implements a "Nearest Reachable Flag" strategy. Our algorithm calls the `NavfnROS/make_plan` service for every remaining flag at each decision point.

1. **Path Validation:** We reject empty plans and also treat a plan as suspicious if its discretized path length is *significantly* smaller than the Euclidean distance (which can indicate stale frames or an invalid plan).
2. **Costmap Synchronization:** We invoke the `clear_costmaps` service before every planning round to reduce the impact of stale obstacle data.
3. **Verifiable Selection:** At each decision point we log the estimated path length to every remaining flag and the selected target, making the nearest-reachable choice easy to verify from terminal output.
4. **Dynamic Targeting:** Users select the 5 flag coordinates interactively in RViz using the `Publish Point` tool, which are then visualized as persistent markers.

1.5 Architecture Diagram

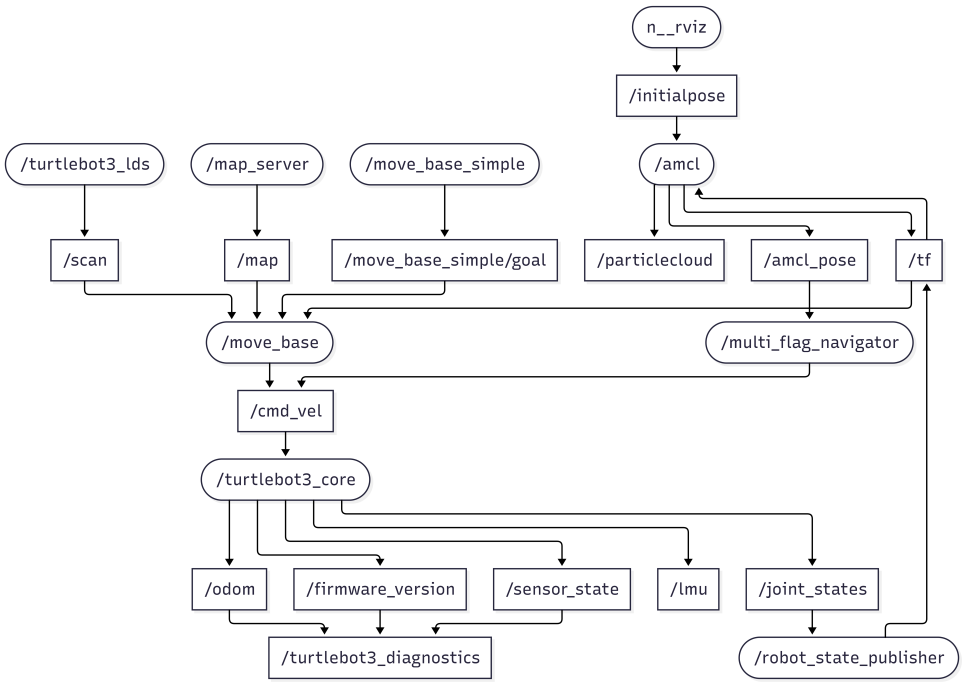


Figure 1: ROS Nodes and Topics Data Flow Architecture

2 Team Contributions

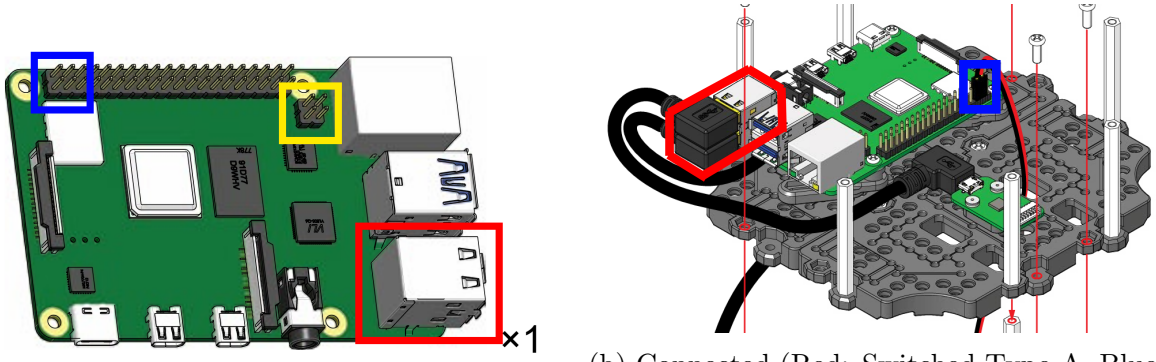
The project roles were assigned to ensure every member contributed to the technical foundation:

- **U2210013 Fuzaylkhon Abdurakhimov:** *Systems Admin.* Configured bridged networking and resolved LDS-02 driver compatibility issues.
- **U2210077 Gayday Aleksey:** *Reliability Engineer.* Implemented approach-offset recovery and costmap-clearing behaviors for stuck states.
- **U2210086 Inomjonov Javohirbek:** *Algorithm Developer.* Implemented the core `flag_manager` logic and the navigable-path distance calculations.
- **U2210088 Sarvar Islamov:** *Team Leader.* Coordinated project tasks, managed the submission process, did dozens of parameter tinkering to find the most optimal ones and ensured requirement compliance.
- **U2210099 Kalimullin Rinat:** *HMI Programmer.* Developed the RViz-based interactive flag selection and persistent markers.
- **U2210122 Shovkatjon Komilov:** *System Architect.* Managed the overall integration of ROS nodes and the coordination between localization and navigation.
- **U2210251 Javohirbek Xatamov:** *Hardware & Paperwork.* Analyzed the physical assembly, diagnosed OpenCR-to-RPi wiring faults, prepared the report.

3 Challenges and Solutions

3.1 Hardware Initialization and Power Delivery

A major early hurdle was the Raspberry Pi 4B failing to boot when powered through the OpenCR 1.0 board. By cross-referencing the official TurtleBot3 manual and using a multimeter, we identified multiple wiring faults. The USB Type-A cables connecting the OpenCR and LiDAR were plugged into the wrong sockets (switched with each other), and the main power cable was connected to an incorrect pin on the Raspberry Pi GPIO header.



(a) RPi Header (Yellow: Wrong Pin)

(b) Connected (Red: Switched Type-A, Blue: Power Tip)

Figure 2: Hardware debugging of the OpenCR to Raspberry Pi power interface. We identified the incorrectly used pin (yellow square) and properly routed the power cable tip (blue square) while correcting the switched USB connections (red square).

3.2 Network and Driver Conflicts

Our development environment initially suffered from high latency and "Master not found" errors. We discovered that the Virtual Machine's NAT network was isolating ROS topics. Switching to **Bridged Networking** allowed the robot and laptop to communicate as peers. Additionally, we fixed a "dead" LiDAR issue by identifying that our Burger model used an LDS-02 sensor, which required the `ld08_driver` instead of the default LDS-01 package.

3.3 The "Wall-Stuck" Problem

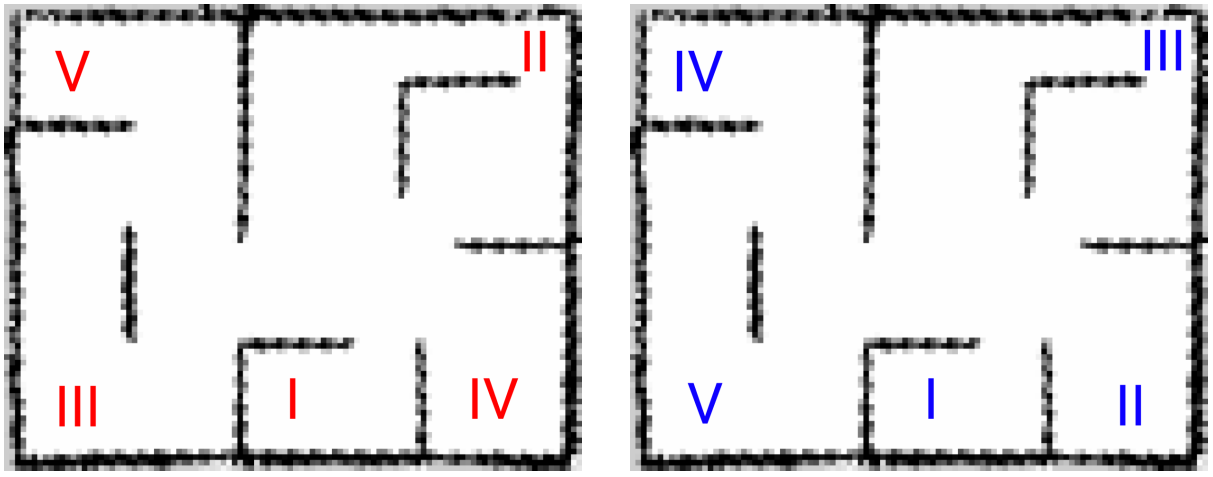
During testing, the robot would frequently freeze when a flag was placed near a wall. The local planner would calculate that any turn would result in a collision. We implemented two key solutions:

1. **Approach Offsets:** We added a function `get_approach_pose()` that calculates a point 10cm away from the actual flag toward the robot, ensuring the goal is in free space while remaining well within the required capture zone.
2. **Capture Radius:** We set the capture threshold to 0.3m, strictly adhering to the project guidelines, which satisfies the "reach the flag" requirement while allowing the robot to safely stop before hitting walls.

4 Results and Performance Analysis

The TurtleBot3 successfully autonomously navigated the maze and captured all 5 flags.

- **Total Mission Time:** 4 minutes and 42 seconds.
- **Efficiency:** By using shortest path distance, the robot avoided long "detours".
- **Stability:** The robot exhibited zero collisions and zero "oscillation" failures after parameter tuning.



(a) Points Selected by Professor (In Sequence) (b) Points Reached by Robot (Nearest Path)

Figure 3: Comparison of the assigned goal sequence (left) versus the optimal path sequence dynamically executed by the robot based on the guidelines (right).

4.1 Flag Capture Order

Per the project requirement, we recorded the order of captured flags from our mission logs.

- **Capture radius:** 0.3m.
- **Captured order (successful run):** F1 → F4 → F2 → F5 → F3

extbfCapture #	**Flag ID**
1	F1
2	F4
3	F2
4	F5
5	F3

Table 1: Flag capture order from the logged run.

5 Future Enhancements

While successful, the system could be further improved by implementing:

- **Dynamic Re-Planning:** Re-evaluating the nearest flag during transit if a more efficient path is discovered as the map updates.
- **SLAM Optimization:** Using Cartographer instead of Gmapping for faster and more precise map updates in feature-less corridors.

6 References

- RViz: `https://wiki.ros.org/rviz`
- Map Server: `https://wiki.ros.org/map_server`
- AMCL: `https://wiki.ros.org/amcl`
- Costmap 2D: `https://wiki.ros.org/costmap_2d`
- move_base: `https://wiki.ros.org/move_base`

- Navigation Stack: https://index.ros.org/p/navigation/
- TurtleBot3 Hardware: https://emanual.robotis.com/docs/en/platform/turtlebot3/hardware_setup/
- TB3 Navigation: https://emanual.robotis.com/docs/en/platform/turtlebot3/navigation/
- TB3 SLAM: https://emanual.robotis.com/docs/en/platform/turtlebot3/slam/
- TB3 Operation: https://emanual.robotis.com/docs/en/platform/turtlebot3/basic_operation/
- MakeNavPlan API: https://docs.ros.org/en/jade/api/navfn/html/srv/MakeNavPlan.html
- Cartographer Docs: https://google-cartographer-ros.readthedocs.io/en/latest/compilation.html